CS422 Assignment - 2 Group - 6

Aman Dixit	Aditya Gautam	Abhijeet Agarwal	Saaumitra Raaj
230112	220064	210025	220928

April 5, 2026

Dynamic Instrumentation and Side-Channel Analysis of RSA Square and Multiply

---

1 Introduction and Objective

The goal of this assignment was to break a vulnerable implementation of RSA using dynamic instrumentation with the help of the Intel PIN tool. The provided 64-bit binary, `leaky_rsa`, utilizes a "Square and Multiply" algorithm for its exponentiation operations.

This specific implementation is highly vulnerable to side-channel analysis. While the "square" operation is executed for every single bit of the key, the "multiply" operation is executed conditionally—only when the corresponding key bit is equal to `1`. By tracing the executed instructions, this differential execution path can be leveraged to figure out the key used for decryption. Our objective was to extract this 64-bit secret key.

2 Phase 1: Instrumentation Setup

To analyze the execution path, we developed an Intel PIN tool designed to log the instruction pointer (IP) and the disassembled mnemonic of every instruction executed by the main binary. To prevent trace pollution, we explicitly filtered out instructions from shared libraries (like `libc`).

`rsa_attack.cpp` (The PIN Tool)

```
#include "pin.H"
#include <iostream>
#include <fstream>
#include <string>

std::ofstream TraceFile;

// Function to print the instruction pointer and the disassembled instruction
VOID PrintInst(ADDRINT ip, std::string* disass) {
    TraceFile << std::hex << ip << " : " << *disass << std::endl;
}

// Pin calls this function every time a new instruction is encountered
VOID Instruction(INS ins, VOID *v) {
    // We only care about instructions in the main executable (leaky_rsa)
    // This prevents our trace from being flooded with standard library setup
   operations.
```

```
    RTN rtn = INS_Rtn(ins);
    if (RTN_Valid(rtn)) {
        SEC sec = RTN_Sec(rtn);
        if (SEC_Valid(sec)) {
            IMG img = SEC_Img(sec);
            if (IMG_Valid(img) && IMG_IsMainExecutable(img)) {
                // Get the disassembled instruction string
                std::string* disass = new std::string(INS_Disassemble(ins));

                // Insert a call to PrintInst before every instruction in the
   main binary
                INS_InsertCall(ins, IPOINT_BEFORE, (AFUNPTR)PrintInst,
                               IARG_INST_PTR,
                               IARG_PTR, disass,
                               IARG_END);
            }
        }
    }
}

// This function is called when the application exits
VOID Fini(INT32 code, VOID *v) {
    TraceFile.close();
}

int main(int argc, char * argv[]) {
    // Initialize PIN library. Print help message if -h(elp) is specified
    if (PIN_Init(argc, argv)) {
        std::cerr << "Command line error" << std::endl;
        return -1;
    }

    // Initialize symbols to safely access image information
    PIN_InitSymbols();

    // Open the output file
    TraceFile.open("trace.txt");

    // Register Instruction to be called to instrument instructions
    INS_AddInstrumentFunction(Instruction, 0);

    // Register Fini to be called when the application exits
    PIN_AddFiniFunction(Fini, 0);

    // Start the program, never returns
    PIN_StartProgram();

    return 0;
}
```

3 Phase 2: Compilation and Debugging Process

During the compilation of the PIN tool, we encountered two environment-specific bugs that required manual intervention.

Bug 1: Stricter GCC Template Checking

Issue: The compilation failed with a fatal error in one of Intel PIN's internal header files (`../../../extras/components/include/util/range.hpp:102`). The newer GCC compiler

caught a typo (`m_base` instead of `_base`) inside the `RANGE` class template. Because PIN compiles with `-Werror`, this strict template warning halted the build.

Resolution: We manually edited the PIN framework source code using `nano`. We navigated to line 102 of `range.hpp` and corrected the typo:

- *Original:* `Contains(const RANGE& range) const { return (Contains(range.m_base) && !range.Contains(GetEnd())); }`
- *Fixed:* `Contains(const RANGE& range) const { return (Contains(range._base) && !range.Contains(GetEnd())); }`

Bug 2: Missing Output Directory

Issue: After fixing the header, the compiler threw: `Fatal error: can't create obj-intel64/rsa_attack. No such file or directory`. The makefile did not automatically generate the target directory for the compiled object files.

Resolution: We manually created the required directory and successfully compiled the tool.

```
mkdir obj-intel64
make obj-intel64/rsa_attack.so TARGET=intel64
```

4 Phase 3: Trace Analysis and Vulnerability Discovery

We executed the PIN tool against the target binary to generate the instruction log:

```
../../../pin -t obj-intel64/rsa_attack.so -- ./leaky_rsa
```

To locate the core "Square and Multiply" loop within the massive `trace.txt` file, we used `grep` to search for multiplication instructions:

```
grep -n -A 10 -B 10 "mul " trace.txt
```

The Assembly Breakdown

Analyzing the resulting assembly blocks revealed the exact mechanics of the vulnerability.

```
47387-7403afc4b780 : mov rax, rsi
47388-7403afc4b783 : xor edx, edx
47389:7403afc4b785 : imul rax, rsi       <-- The constant "Square" operation
47390-7403afc4b789 : div rcx
47391-7403afc4b78c : bt r9, rdi          <-- The Vulnerability: Bit Test
47392-7403afc4b790 : mov rsi, rdx
47393-7403afc4b793 : jnb 0x7403afc4b7a4  <-- The Conditional Jump
...
47863:7403afc4b795 : imul rax, rdx, 0x357 <-- The conditional "Multiply"
    operation
...
47394-7403afc4b7a4 : sub edi, 0x1        <-- The Loop Counter Decrement
47395-7403afc4b7a7 : jnb 0x7403afc4b780  <-- Loop back to start
```

Key Discoveries:

1. **The Loop Counter (`rdi`):** The register `rdi` is used as the loop counter, decrementing by 1 at the end of each cycle (`sub edi, 0x1`).
2. **The Secret Key (`r9`):** The `bt r9, rdi` (Bit Test) instruction tests the bit of register `r9` at the position specified by the counter `rdi`. This confirms `r9` holds the 64-bit secret key.
3. **The Leak (`jnb`):** The `jnb` (Jump if Not Below / Jump if Carry Flag is 0) instruction dictates the flow. If the tested bit is `0`, the program jumps to `0x7403afc4b7a4`, completely bypassing the multiply phase. If the bit is `1`, it falls through and executes the `imul` instruction at `0x7403afc4b795`.

5 Phase 4: Automated Key Extraction

To build a robust exploit that bypasses Address Space Layout Randomization (ASLR) and avoids hardcoding specific memory addresses or ciphertext operands, we designed an ASLR-proof Python parsing script and an automated bash wrapper. Instead of relying on static addresses, the Python script analyzes the topological behavior of the execution trace, recording a `1` whenever the number of multiplication operations between loop boundaries (`bt` instructions) exceeds one.

`auto_crack.sh` (The Automation Wrapper)

```
#!/bin/bash

# Define paths and filenames
PIN_BIN="../../../pin"
PIN_TOOL="obj-intel64/rsa_attack.so"
PYTHON_EXTRACTOR="extract_key.py"
TRACE_FILE="trace.txt"
KEY_FILE="key.txt"

# Check if the user provided a target binary
if [ -z "$1" ]; then
    echo "Usage: ./auto_crack.sh <path_to_vulnerable_binary>"
    exit 1
fi

TARGET_BIN="$1"

echo "================================================"
echo "          RSA Square & Multiply Auto-Cracker     "
echo "================================================"

# Step 1: Clean up old artifacts
rm -f $TRACE_FILE $KEY_FILE

# Step 2: Run the PIN instrumentation
echo "[*] Step 1: Instrumenting target binary to generate execution trace..."
$PIN_BIN -t $PIN_TOOL -- $TARGET_BIN > /dev/null 2>&1

if [ ! -f "$TRACE_FILE" ]; then
    echo "[-] Error: Failed to generate trace file. Check your PIN paths and
    compilation."
    exit 1
fi
echo "[+] Trace generated successfully."

# Step 3: Run the Python parsing script
echo "[*] Step 2: Parsing trace to extract 64-bit secret key..."
python3 $PYTHON_EXTRACTOR > /dev/null

if [ ! -f "$KEY_FILE" ]; then
    echo "[-] Error: Python script failed to generate key.txt."
    exit 1
fi

# Step 4: Verify the key against the target binary
EXTRACTED_KEY=$(cat $KEY_FILE)
echo "[+] Key extracted: $EXTRACTED_KEY"
echo "[*] Step 3: Verifying key against the target binary..."
echo "------------------------------------------------"
```

```
$TARGET_BIN -a $EXTRACTED_KEY

echo "------------------------------------------------"
echo "[+] Exploit sequence complete."
```

extract_key.py (The Parsing Script)

```
import sys

def extract_key(trace_file):
    print(f"[*] Analyzing execution trace: {trace_file}...")

    bits = []
    mul_count = 0
    in_loop = False

    with open(trace_file, 'r') as f:
        for line in f:
            # The 'bt' (Bit Test) instruction acts as our loop boundary
            if " bt " in line:
                if in_loop:
                    # If we counted more than 1 multiply, the conditional
   branch executed!
                    if mul_count > 1:
                        bits.append('1')
                    else:
                        bits.append('0')

                in_loop = True
                mul_count = 0 # Reset counter for the next loop iteration

            # Count every multiplication operation within the current loop
   boundary
            elif ("imul " in line or " mul " in line) and in_loop:
                mul_count += 1

    # Evaluate the very last loop iteration after the file ends
    if in_loop:
        if mul_count > 1:
            bits.append('1')
        else:
            bits.append('0')

    # The algorithm works from MSB to LSB. Isolate the final 64 bits.
    if len(bits) > 64:
        bits = bits[-64:]

    binary_string = "".join(bits)

    try:
        decimal_key = int(binary_string, 2)
    except ValueError:
        print("[-] Error: Could not extract a valid binary sequence.")
        sys.exit(1)

    print(f"[*] Loop iterations analyzed: {len(bits)}")
    print(f"[*] Extracted Binary Key  : {binary_string}")
    print(f"[*] Extracted Decimal Key : {decimal_key}")

    with open("key.txt", "w") as kf:
        kf.write(str(decimal_key))
    print("[+] Successfully wrote the extracted key to key.txt")
```

```
if __name__ == "__main__":
    extract_key("trace.txt")
```

6 Conclusion and Verification

Running the automated extraction sequence executed PIN, successfully generated the trace, and bypassed ASLR to robustly reconstruct the target key:

- **Extracted Binary:** `000000000000000000000000000000000000000000000000000101011000001`
- **Extracted Decimal:** `2753`

To verify our findings and streamline the attack, we executed the wrapper script against the target binary:

```
./auto_crack.sh ./leaky_rsa
```

The wrapper passed the extracted decimal key into the target binary (utilizing the `-a` flag), and the binary returned the `Your Key is Correct` message. This confirmed the successful, automated execution of the side-channel attack and the extraction of the 64-bit private key independent of hardcoded execution addresses or ciphertext values.

References

- CS422 Assignment 2 Problem Statement
- Intel PIN Tool Documentation